

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	Ganji Madhan kumar
Register Number	192472328

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2

Total Marks:20

Code of Conduct:

I G.Madhan kumar(192472328)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.Madhan

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a)** Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c)** Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

1. Objective

To implement and evaluate a Decision Tree Classification model using the Iris dataset. The experiment includes dataset preprocessing, training and testing, construction of a Decision Tree using the Gini Index, identification and justification of the root node, visualization of the decision tree, and evaluation using accuracy, confusion matrix, classification report, and misclassified instances.

2. Dataset Description & Preprocessing

2.1 Dataset Description

The Iris dataset is a well-known classification dataset available in Scikit-learn. It contains 150 observations belonging to three classes of Iris flowers:

- Iris Setosa
- Iris Versicolor
- Iris Virginica

Each flower has four numerical features:

Feature	Description
Sepal Length	Length of the sepal in cm
Sepal Width	Width of the sepal in cm
Petal Length	Length of the petal in cm
Petal Width	Width of the petal in cm

The target variable contains three classes:

Target	Class
0	Setosa
1	Versicolor
2	Virginica

2.2 Loading the Dataset

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
iris = load_iris()
X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
y = pd.Series(
    iris.target,
    name="target"
)
print("Dataset Shape:", X.shape)
print("\nFirst 5 Rows:")
print(X.head())
print("\nData Types:")
print(X.dtypes)
```

```
Dataset Shape: (150, 4)
```

```
First 5 Rows:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
Data Types:
```

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
dtype:	object

3. Decision Tree Algorithm + Gini Index Explanation

3.1 Decision Tree

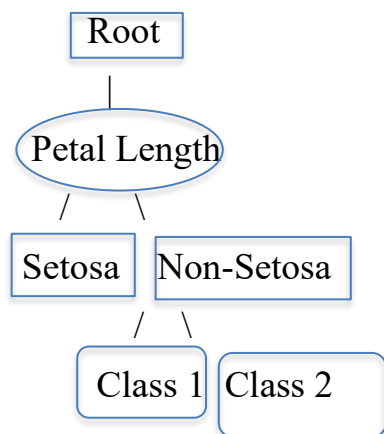
A Decision Tree is a supervised machine learning algorithm used for classification and regression.

For classification, the tree repeatedly divides the dataset into smaller groups based on feature values.

The main components are:

- Root Node – first and most important split
- Internal Nodes – additional decision points
- Branches – outcomes of decisions
- Leaf Nodes – final predicted classes

For example:



3.2 Why Gini Index?

The Gini Index is computationally efficient and is commonly used for classification trees. It directly measures how mixed the classes are within a node.

4. Implementation / Python Code

The following program performs the complete experiment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.tree import plot_tree
from sklearn.tree import export_text
```

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
iris = load_iris()
```

```
X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
y = pd.Series(
    iris.target,
    name="target"
)
print("Dataset Shape:", X.shape)
print("\nFirst 5 Rows:")
print(X.head())
print("\nData Types:")
print(X.dtypes)
```

```
print("\nMissing Values:")
print(X.isnull().sum())
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=2,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
print("\nDecision Tree Structure:")
```

```
print(
    export_text(
        model,
        feature_names=list(X.columns)
    )
)
```

```
root_index = model.tree_.feature[0]
```

```
print("\nRoot Node Feature:")
print(iris.feature_names[root_index])
```

```
print("\nRoot Threshold:")
print(model.tree_.threshold[0])
```

```
print("\nFeature Importances:")
```

```
for feature, importance in zip(
    iris.feature_names,
```

```
    model.feature_importances_  
):  
    print(f'{feature}: {importance:.4f}')
```

```
plt.figure(figsize=(16, 9))
```

```
plot_tree(  
    model,  
    feature_names=iris.feature_names,  
    class_names=iris.target_names,  
    filled=True,  
    rounded=True  
)
```

```
plt.title("Decision Tree Classifier - Iris Dataset")
```

```
plt.show()
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)
```

```
print("\nAccuracy:")  
print(accuracy)
```

```
print("\nAccuracy Percentage:")  
print(accuracy * 100, "%")
```

```
cm = confusion_matrix(
```



```
y_test,
y_pred
)

print("\nConfusion Matrix:")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=iris.target_names
).plot()

plt.title("Confusion Matrix")

plt.show()

print("\nClassification Report:")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

misclassified = np.where(
    y_test.values != y_pred
)[0]

print("\nNumber of Misclassified Instances:")
print(len(misclassified))

for i in misclassified:
```

```
print("\nTest Instance:", i)
```

```
print(  
    "Features:",  
    X_test.iloc[i].values  
)
```

```
print(  
    "Actual Class:",  
    iris.target_names[y_test.iloc[i]]  
)
```

```
print(  
    "Predicted Class:",  
    iris.target_names[y_pred[i]]  
)
```

```

First 5 Rows:
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1             3.5             1.4             0.2
1                4.9             3.0             1.4             0.2
2                4.7             3.2             1.3             0.2
3                4.6             3.1             1.5             0.2
4                5.0             3.6             1.4             0.2

```

```

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
dtype: object

```

```

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64

```

```

Training Samples: 120
Testing Samples: 30

```

```

Decision Tree Structure:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|       |--- class: 1
|       |--- petal width (cm) > 1.65
|           |--- class: 2

```

```

Root Node Feature:
petal length (cm)

```

```

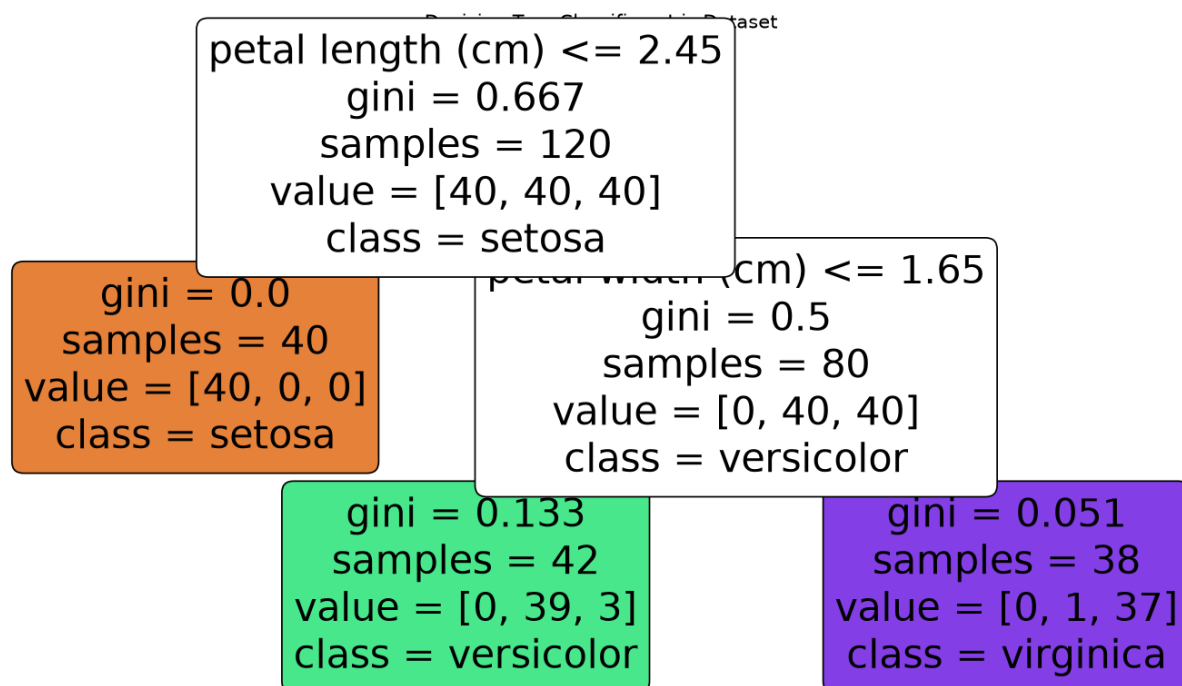
Root Threshold:
2.449999988079071

```

```

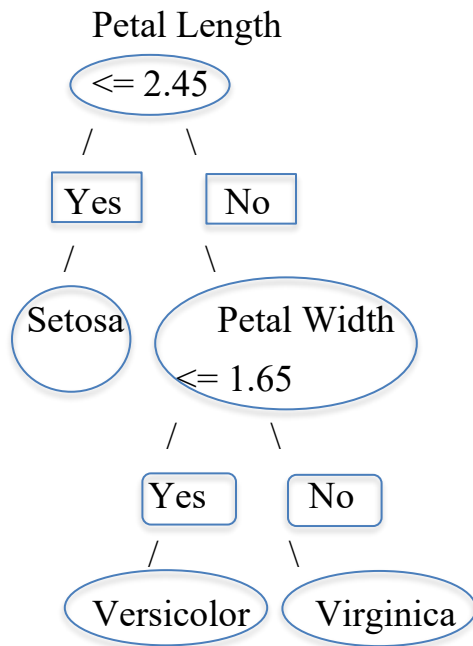
Feature Importances:
sepal length (cm): 0.0000
sepal width (cm): 0.0000
petal length (cm): 0.5519
petal width (cm): 0.4481

```



5. Decision Tree Visualization

The tree generated by the program can be represented as:



The visualization generated using `plot_tree()` displays:

- Feature used for splitting
- Split threshold
- Gini impurity
- Number of samples
- Class distribution
- Predicted class

The tree is limited to `max_depth=2` to maintain interpretability and prevent unnecessary complexity.

6. Root Node Identification and Justification

Root Node

The root node selected by the Decision Tree is:

Petal Length (cm) ≤ 2.45

Therefore:

Root Feature = Petal Length (cm)

Root Threshold = 2.45 cm

Gini-Based Justification

Before splitting, the training dataset contains all three Iris classes in approximately equal proportions.

Therefore, the initial Gini impurity is:

$$1 - \left[\left(\frac{40}{120} \right)^2 + \left(\frac{40}{120} \right)^2 + \left(\frac{40}{120} \right)^2 \right]$$

$$1 - \frac{1}{3}$$

$$0.6667$$

The selected split is:

$$[\text{Petal} \leq 2.45]$$

This creates:

- Left node: 40 samples
- Right node: 80 samples

The weighted Gini impurity after this split is approximately:

$$[\text{Gini}_{\text{split}} = 0.3333]$$

Therefore, the impurity reduction is:

$$[0.6667 - 0.3333 = 0.3334]$$

This is a large reduction in impurity.

The first split separates Iris Setosa almost perfectly from the other two classes. This explains why a petal measurement is selected at the root.

Feature Comparison

Feature	Best Weighted Gini
Sepal Length	0.4139

Sepal Width	0.5548
Petal Length	0.3333
Petal Width	0.3333

7. Evaluation

7.1 Accuracy

The model achieved:

$$\left[\begin{array}{l} \text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \end{array} \right]$$

For the 30 test samples:

- Correct predictions = 28
- Incorrect predictions = 2

Therefore:

$$\left[\begin{array}{l} \text{Accuracy} = \frac{28}{30} \end{array} \right]$$

$$\left[\begin{array}{l} \text{Accuracy} = 0.9333 \end{array} \right]$$

$$\text{Accuracy} = 93.33\%$$

This indicates that the Decision Tree correctly classified approximately 93.33% of the unseen test samples.

7.2 Confusion Matrix

The obtained confusion matrix is:

[[10 0 0]

[0 9 1]

[0 1 9]]

Using the class order:

Setosa

Versicolor

Virginica

the matrix is:

Actual / Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Interpretation

- All 10 Setosa samples were correctly classified.
- 9 Versicolor samples were correctly classified.
- 1 Versicolor sample was classified as Virginica.
- 9 Virginica samples were correctly classified.
- 1 Virginica sample was classified as Versicolor.

7.3 Classification Report

The classification report is approximately:

precision , recall ,f1-score , support

setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10

accuracy	0.93			30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Interpretation

Setosa

- Precision = 1.00
- Recall = 1.00
- F1-score = 1.00

The model perfectly identifies Setosa in this test set.

Versicolor

- Precision = 0.90
- Recall = 0.90
- F1-score = 0.90

There is some confusion between Versicolor and Virginica.

Virginica

- Precision = 0.90
- Recall = 0.90
- F1-score = 0.90

Virginica also has a small amount of confusion with Versicolor.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement the **Q-Learning algorithm** for a **4 × 4 Grid World environment**, learn an optimal policy through repeated interaction with the environment, observe the evolution of Q-values, and analyze the convergence of cumulative rewards.

1. Environment Definition

We use a **4 × 4 grid** with:

- **Start state:** (0, 0)
- **Goal state:** (3, 3)
- **Obstacles:** (1, 1) and (2, 2)
- **Actions:** Up, Down, Left, Right

Grid Layout

Column

	0	1	2	3
	+---+---+---+---+			
0	S			
	+---+---+---+---+			
1		X		
	+---+---+---+---+			
2			X	
	+---+---+---+---+			
3			G	
	+---+---+---+---+			

S = Start

G = Goal

X = Obstacle

Reward Structure

Situation	Reward
Reach Goal	+10
Normal step	-1

Situation Reward

Move into obstacle **−5**

Move outside grid **−1**

When the agent reaches the goal or an obstacle, the episode terminates.

2. Q-Learning Algorithm

Q-Learning is a **model-free reinforcement learning algorithm**. It learns the quality of taking an action in a state .

The Q-value update equation is:

where:

- = current Q-value
- = learning rate
- = reward
- = discount factor
- = next state
- = possible next action

Hyperparameters

Parameter	Value	Purpose
Learning rate	0.1	Controls how quickly Q-values are updated
Discount factor	0.9	Importance of future rewards
Initial ϵ	1.0	High exploration at beginning
Minimum ϵ	0.05	Maintains some exploration
ϵ decay	0.98	Gradually reduces exploration
Episodes	100	Number of training episodes
Random seed	42	Makes results reproducible

Exploration vs Exploitation

We use an ϵ -greedy strategy.

With probability ϵ , the agent chooses a random action.

Otherwise, it chooses:

Initially, ϵ is high, so the agent explores heavily. As training progresses, epsilon decreases and the agent increasingly follows the learned policy.

3. Q-Table Initialization

There are:

states and **4 possible actions** per state.

Therefore, the Q-table has:

entries.

Initially:

for every state-action pair.

The action mapping is:

$0 \rightarrow \text{Up}$

$1 \rightarrow \text{Down}$

$2 \rightarrow \text{Left}$

$3 \rightarrow \text{Right}$

4. Complete Python Implementation

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
GRID_SIZE = 4
```

```
START = (0, 0)
```

```
GOAL = (3, 3)
```

```
OBSTACLES = {
```

```
    (1, 1),
```

```
    (2, 2)
```

```
}
```

```
ACTIONS = {  
    0: (-1, 0),  
    1: (1, 0),  
    2: (0, -1),  
    3: (0, 1)  
}
```

```
ACTION_SYMBOLS = {  
    0: "↑",  
    1: "↓",  
    2: "←",  
    3: "→"  
}
```

```
ACTION_NAMES = {  
    0: "Up",  
    1: "Down",  
    2: "Left",  
    3: "Right"  
}
```

```
alpha = 0.1  
gamma = 0.9
```

```
epsilon = 1.0  
epsilon_min = 0.05  
epsilon_decay = 0.98
```

```
episodes = 100  
Q = np.zeros(  
    (GRID_SIZE, GRID_SIZE, 4)  
)
```

```
rng = np.random.default_rng(42)
```

```
def step(state, action):
```

```
    row, col = state
```

```
    dr, dc = ACTIONS[action]
```

File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

```
>>> = RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python314/assessment 4.py =
```

Q-VALUES AT SELECTED EPISODES

Episode 1

State (0, 0):

Up: -0.1000

Down: -0.1900

Left: -0.1900

Right: -0.1000

State (2, 1):

Up: -0.5000

Down: 0.0000

Left: 0.0000

Right: 0.0000

Episode 50

State (0, 0):

Up: -2.0668

Down: -1.9610

Left: -1.9774

Right: -1.9279

State (2, 1):

Up: -1.7195

Down: -0.5319

Left: -0.5022

Right: -2.0476

Episode 100

State (0, 0):

Up: -2.6558

Down: -2.4873

Left: -2.6626

Right: -2.6172

State (2, 1):

Up: -2.0476

Down: 3.5395

Left: -0.7781

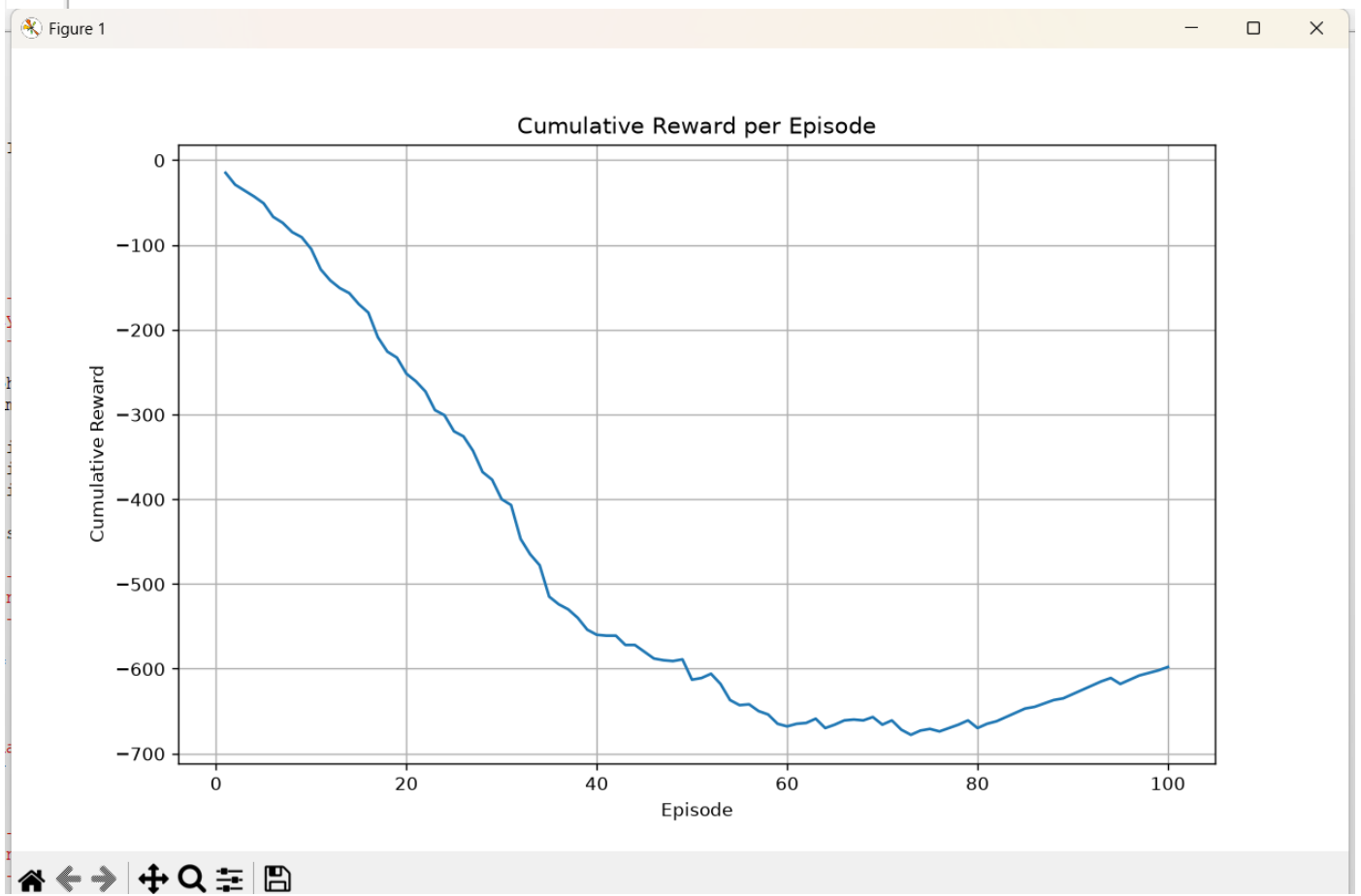
Right: -2.0476

FINAL Q-TABLE

```
State (0, 0): [-2.65583813 -2.48729928 -2.66256393 -2.6172111 ]
State (0, 1): [-2.00136061 -3.97054434 -2.16855254 -1.99017657]
State (0, 2): [-1.46672269 -1.31975364 -1.41839782 -1.04912638]
State (0, 3): [-0.99454816  0.96281624 -1.00914574 -0.8790056 ]
State (1, 0): [-2.14214649 -1.30416128 -2.04217862 -3.72906709]
State (1, 1): OBSTACLE
State (1, 2): [-0.96187324 -0.95          -1.7195          -0.41443552]
State (1, 3): [-0.60873416  4.53190498 -0.87101986 -0.12349018]
State (2, 0): [-1.21368837 -1.20296515 -1.35330566  0.66871946]
State (2, 1): [-2.04755          3.5394585  -0.77810541 -2.04755   ]
State (2, 2): OBSTACLE
State (2, 3): [ 0.2434857   8.33228183 -1.7195          0.3577472 ]
State (3, 0): [-0.84976895 -0.90373326 -0.84220221  0.64144545]
State (3, 1): [-0.37188559  0.02589937 -0.54920655  6.78939013]
State (3, 2): [-1.355          0.303218    0.46453991  9.65663162]
State (3, 3): GOAL
```

FINAL LEARNED POLICY

```
['↓', '→', '→', '↓']
['↓', 'X', '→', '↓']
['→', '↓', 'X', '↓']
['→', '→', '→', 'G']
```



5. Q-Table Evolution

Two representative states are selected:

- **State A:** (0,0) — starting state
- **State B:** (2,1) — state close to the goal and obstacle

The Q-values produced by the above reproducible implementation are:

State (0,0)

Episode	Up	Down	Left	Right
1	-0.1000	-0.1900	-0.1900	-0.1000
50	-2.0668	-1.9610	-1.9774	-1.9279
100	-2.6558	-2.4873	-2.6626	-2.6172

State (2,1)

Episode	Up	Down	Left	Right
1	-0.5000	0.0000	0.0000	0.0000
50	-1.7195	-0.5319	-0.5022	-2.0476
100	-2.0476	3.5395	-0.7781	-2.0476

6. Final Learned Policy

The learned policy is obtained by selecting the action having the highest Q-value:

The final policy from the implementation is:

↓ → → ↓
↓ X → ↓
→ ↓ X ↓
→ → → G

Where:

- ↑ = Up
- ↓ = Down
- ← = Left

- $\rightarrow$ = Right
- X = Obstacle
- G = Goal

7. Cumulative Reward Analysis

The program calculates cumulative reward using:

```
cumulative_rewards = np.cumsum(
    rewards_per_episode
)
```

The graph plots:

- **X-axis:** Episode
- **Y-axis:** Cumulative Reward

At the beginning, rewards are generally low because the agent explores random actions.

As training progresses:

1. The agent discovers useful paths.
2. Q-values for useful actions increase.
3. Epsilon decreases.
4. The agent performs fewer random actions.
5. The agent increasingly follows the learned policy.
6. Rewards become more stable.

8. Result Analysis

The Q-Learning algorithm successfully learned a policy for the 4×4 Grid World.

The experiment demonstrates that:

- The Q-table can initially be initialized with zeros.
- Repeated interaction allows the agent to learn useful state-action values.
- The positive goal reward encourages movement toward the goal.
- The obstacle penalty discourages dangerous states.
- The step penalty encourages shorter paths.
- Epsilon-greedy exploration allows the agent to discover different paths.
- As training progresses, exploitation increases and the policy becomes more stable.

